

PC SYSTEM DASHBOARD

Project #1 - Detailed Build & Behavior Specification

A small, polished Windows-first desktop utility designed to test the complete CSE project workflow.

Primary Objective

Finish a real project. The point is not maximum technical complexity; the point is to prove the system can take an idea from scope -> build -> debug -> polish -> documentation -> portfolio-ready completion.

Target difficulty: 3.5/10 | Expected effort: ~5-8 focused hours | Platform: Windows-first

Recommended stack: Python + psutil + Tkinter

1. Project Summary

Build a lightweight desktop application that displays the current state of the user's computer in a clean dashboard. The first release should show only the metrics that are easy to collect reliably and present them in an intentional interface.

Field	Decision
Project name	PC System Dashboard
Project type	Small portfolio utility / systems-monitoring desktop app
Target user	You first; later, anyone who wants a simple at-a-glance system monitor
Target OS	Windows 10/11 for v1
Primary language	Python
Core library	psutil
GUI	Tkinter / ttk
Storage	None required for MVP
Network access	None
Account/login	None
MVP metrics	CPU usage, memory usage, disk usage, uptime
Stretch metrics	CPU temperature if available, network throughput, GPU data, process list
Definition of success	A clean executable program that starts reliably, refreshes live metrics, handles unavailable data gracefully, and is documented well enough for another person to run.

2. Why This Is the Right First Project

- It is small enough to finish quickly, so the workflow itself can be tested instead of spending weeks inside one build.
- It has visible output, which makes screenshots and portfolio presentation easy.
- It introduces a new language/library without requiring a large framework.
- It exercises real engineering behaviors: requirements, dependency setup, live data collection, GUI state updates, error handling, testing, packaging, documentation, and polish.
- It has a very clear stopping point. Once the MVP is stable and polished, it is done.
- It can be expanded later, but expansion is explicitly not required for Project #1.

MVP FIRST

Do not add GPU monitoring, charts, process management, remote monitoring, accounts, cloud storage, or AI before the four core metrics work and the application is already presentable.

3. Product Behavior - What the App Should Feel Like

The application should feel like a small utility somebody intentionally designed, not a programming exercise. The behavior should be immediate, predictable, and visually calm.

Situation	Expected Behavior
Launch	Double-clicking the app opens one main window. No terminal window should be required in the final packaged version.
Initial state	The window appears quickly. Metric cards may briefly show "Loading..." until the first sample is available.
Live refresh	CPU, memory, and disk values update automatically. Recommended refresh interval: once per second.

Uptime	Uptime updates at least once per minute or can update with the normal refresh loop.
No user setup	The user should not need to configure paths, API keys, accounts, or settings for the MVP.
Failure behavior	If a metric cannot be read, that card shows a clear neutral fallback such as “Unavailable” instead of crashing the program.
Close behavior	Closing the main window stops the refresh loop and exits cleanly.
Resize behavior	The layout should remain readable at the intended default size. Full responsive behavior is optional; minimum window size may be enforced.
Offline behavior	The app works without internet access.
Permissions	Normal user permissions should be sufficient for all MVP functionality.

4. MVP Screen Specification

4.1 Main Window

- Window title: “PC System Dashboard”.
- Default size: approximately 760 x 460 pixels. Exact dimensions may change during polish.
- Minimum size: enough that all four metric cards remain readable.
- Background: neutral/light or neutral/dark; choose one theme and keep it consistent.
- Top area: application title plus a small status line such as “Live system metrics”.
- Primary content: four cards arranged in a 2 x 2 grid.
- Bottom area: small “Last updated” timestamp and optional app version.

4.2 Metric Cards

Card	Primary Value	Secondary Detail	Meaning
CPU	Large percentage, e.g. 27%	Optional progress bar	System-wide CPU utilization
Memory	Used percentage, e.g. 61%	Used / total GB	Physical RAM utilization
Disk	Used percentage, e.g. 72%	Used / total GB	Primary system drive only for MVP
Uptime	Human-readable time, e.g. 1d 4h 12m	Boot time optional	Time since most recent system boot

4.3 Visual Hierarchy

1. User sees the app name first.
2. User immediately sees four large current values.
3. Secondary numbers explain the primary values without competing for attention.
4. Last-updated information is visible but visually quiet.
5. No unnecessary menus, sidebars, tabs, graphs, or settings in v1.

5. Functional Requirements

ID	Requirement	Priority
FR-01	Read system-wide CPU utilization.	Must
FR-02	Read physical memory usage and total memory.	Must
FR-03	Read usage and capacity for the Windows system drive.	Must
FR-04	Determine system boot time and	Must

	calculate uptime.	
FR-05	Display all four metrics in one GUI window.	Must
FR-06	Refresh live metrics automatically without user interaction.	Must
FR-07	Keep the GUI responsive during refresh.	Must
FR-08	Show a safe fallback if a metric read fails.	Must
FR-09	Display a last-updated time.	Should
FR-10	Package the app so it can be started by a non-developer.	Should
FR-11	Show app version somewhere unobtrusive.	Could
FR-12	Allow theme selection.	Won't for MVP

6. Non-Functional Requirements

- Reliability: ordinary metric-read failures must not terminate the application.
- Responsiveness: GUI interactions and window movement should remain responsive while metrics refresh.
- Startup: the app should reach a usable screen within a few seconds on a normal Windows PC.
- Resource usage: the monitor itself should remain lightweight and should not noticeably increase CPU usage.
- Readability: values should be legible at normal Windows scaling.
- Maintainability: metric collection should be separated from presentation logic enough that another metric could be added without rewriting the whole app.
- Privacy: no metrics are transmitted anywhere.
- Portability: source code should run after installing documented dependencies; packaged executable is Windows-first.

7. Recommended Architecture

Keep the architecture deliberately simple. The goal is separation of concerns, not enterprise-style layering.

Three-part model

1) System Metrics layer collects data. 2) Dashboard UI layer owns widgets and presentation. 3) Application entry point initializes the UI and starts the refresh cycle.

File / Folder	Purpose
main.py	Application entry point. Creates the root window, initializes the dashboard, and starts the program.
metrics.py	Functions for CPU, memory, disk, uptime, and formatting. No GUI code.
dashboard.py	Tkinter/ttk widgets, layout, refresh scheduling, and display updates.
requirements.txt	Pinned or minimally specified runtime dependency list.
README.md	Project explanation, setup, screenshots, architecture, and lessons learned.
assets/	Optional icon and screenshots. Keep empty until needed.
tests/	Small tests for deterministic formatting/helper functions if useful.

8. Suggested Repository Layout

```
pc-system-dashboard/  
  README.md  
  requirements.txt  
  .gitignore  
  src/  
    main.py  
    metrics.py  
    dashboard.py  
  tests/  
    test_metrics.py  
  assets/  
    app_icon.ico  
    screenshot.png  
  docs/  
    project-spec.pdf
```

Organization rule

Do not create folders merely because professional repositories sometimes have them. Add only what this project actually uses.

9. Data Collection Rules

Metric	Implementation Behavior
CPU	Use <code>psutil.cpu_percent()</code> . A one-second UI refresh is adequate. Avoid blocking calls inside the GUI loop.
Memory	Use <code>psutil.virtual_memory()</code> . Display percentage and convert used/total bytes to GB.
Disk	Use <code>psutil.disk_usage()</code> for the Windows system drive. Detect or document the drive target rather than silently assuming unusual layouts.
Uptime	Use <code>psutil.boot_time()</code> and subtract from current time. Format into days/hours/minutes.
Timestamp	Use local system time for “Last updated”.

10. Refresh Loop Behavior

- Window launches and creates placeholder values.
- Application requests the first metric snapshot.
- UI updates all cards from that snapshot.
- UI schedules the next refresh approximately 1000 ms later using Tkinter's event scheduling mechanism.
- Each refresh is wrapped so one failed metric does not prevent the remaining metrics from updating.
- When the window closes, no background loop should keep the process alive.

Important implementation constraint

Do not use a `while True` loop in the GUI thread. That would freeze the interface. Use Tkinter `after(...)` scheduling for the MVP.

11. Error Handling Specification

Failure	Required Response
Metric library unavailable at startup	Fail with a readable dependency/setup message during development; packaged build should contain the

	dependency.
One metric call raises an exception	Catch at the metric boundary, return an unavailable state, keep other cards working.
Unexpected disk path issue	Show “Unavailable” on Disk and continue.
Formatting receives invalid data	Return a safe display string instead of throwing into the GUI.
UI update error	Log enough context during development to identify which metric/update failed.
Packaging issue on another machine	Document supported Windows version and test on at least one clean-ish environment if possible.

12. Development Milestones

Milestone	Work	Exit Condition
M0 - Repo Ready	Create repository, README placeholder, Python environment, dependency file, base folder structure.	Project launches a hello-window from source.
M1 - Metrics Work	Implement and manually verify CPU, RAM, disk, uptime functions.	Terminal test prints believable values without exceptions.
M2 - MVP UI	Create four-card dashboard and connect current values.	All required metrics are visible in one window.
M3 - Live Behavior	Add scheduled refresh, last-updated label, and safe fallback states.	App runs 10+ minutes without freezing or crashing.
M4 - Polish	Improve spacing, typography, labels, window sizing, icon, naming, and remove rough/debug UI.	Screenshot looks intentional and readable.
M5 - Test & Package	Run test checklist, clean warnings, build executable if practical.	Another person can launch it using README instructions.
M6 - Portfolio Closeout	Finalize README, screenshot, architecture summary, lessons learned, tagged release.	Project is formally marked finished.

13. Recommended Work Sessions

Session	Target
Session 1 - Foundation (~60 min)	Create repo/environment; install psutil; build metric functions; commit.
Session 2 - Interface (~90 min)	Build the dashboard layout and connect four metrics; commit working MVP.
Session 3 - Reliability (~60 min)	Add refresh scheduling, error states, formatting helpers; run long-lived test.
Session 4 - Polish (~60-90 min)	Spacing, labels, typography, icon, packaging, screenshot.
Session 5 - Closeout (~45 min)	README, setup verification, final tests, release/tag, lessons learned.

14. Git / Commit Expectations

- Commit after meaningful working states, not after every tiny edit.
- Good examples: “add system metric collection”, “build live dashboard cards”, “handle unavailable metrics”, “package Windows executable”.
- Keep main branch runnable whenever practical.
- Do not commit virtual environments, build folders, caches, or generated temporary files.
- Use the project to practice reading diffs before committing.
- If a feature breaks the app, fix or revert rather than leaving main in an unclear state.

15. Testing Plan

Test	Procedure	Pass Condition
Launch test	Start from source 5 times.	Window opens every time with no traceback.
Metric plausibility	Compare dashboard values with Windows Task Manager.	Values are reasonably close; small timing differences are acceptable.
Refresh test	Leave app open for at least 10 minutes.	Metrics continue updating; UI does not freeze.
Load reaction	Open a CPU-heavy program or several tabs.	CPU/RAM values visibly respond.
Resize test	Resize around default/minimum size.	No critical text overlaps or disappears.
Failure simulation	Temporarily force one metric helper to fail.	Only that card becomes unavailable; application remains usable.
Close test	Close via window X.	Process exits cleanly.
Fresh setup test	Follow README setup steps from a clean environment if possible.	Instructions are complete enough to reproduce the app.
Packaged app test	Launch built executable without IDE.	Dashboard works with no development terminal required.

16. Polish Checklist

Functionality

- Every required metric works.
- No visible debug placeholders remain.
- Refresh behavior is stable.

Reliability

- Unavailable metrics do not crash the app.
- App survives extended idle use.
- Close behavior is clean.

User Experience

- Labels are obvious without explanation.
- Values are readable at a glance.
- No unnecessary controls exist.

Design

- Consistent spacing and alignment.
- Consistent typography.
- One coherent theme.
- Cards have equal visual weight.
- Icon and window title are intentional.

Code Quality

- Metric collection and GUI code are separated.
- Names communicate intent.
- Dead code and commented-out experiments are removed.
- Only useful comments remain.

Documentation

- README explains what it is, why it exists, how to run it, stack, architecture, and limitations.
- Screenshot included.
- Dependencies and supported OS are explicit.

17. README Specification

README Section	Required Content
Title + one-line pitch	PC System Dashboard - a lightweight Windows desktop utility for live CPU, memory, disk, and uptime monitoring.
Screenshot	One clean image of the final dashboard.
Features	Four MVP metrics plus automatic refresh and graceful errors.
Why I built it	Explain that it was the first project used to practice a complete engineering workflow and project polish.
Tech stack	Python, psutil, Tkinter/ttk, packaging tool if used.
How to run	Clone, create environment, install requirements, run src/main.py.
Architecture	Short explanation of metrics layer, UI layer, entry point.
Testing	Brief list of manual checks and any automated tests.
What I learned	Concrete observations: GUI event loops, reading OS metrics, packaging, error handling, project finishing.
Limitations	Windows-first, no historical charts, no GPU data, no advanced process management.
Future ideas	Optional list, clearly marked as not part of v1.

18. Definition of Done

The project is DONE only when all of these are true

The application works from source; all four metrics display and refresh; failure of one metric does not crash the app; the UI has received a deliberate polish pass; README and screenshot are complete; setup instructions were tested; repository is clean; final version is tagged/released or otherwise clearly marked; you can explain the architecture and the main problems you solved.

19. Explicitly Out of Scope for Version 1

- GPU temperature/utilization.
- Historical graphs or persistent metric storage.
- Process killing or task-manager functionality.
- Remote monitoring or web dashboard.
- User accounts, cloud sync, or authentication.
- Notifications/alerts.
- AI features.
- Customizable widgets/themes.
- Multi-machine support.
- Cross-platform guarantees.
- Installer engineering beyond what is reasonably needed to demonstrate the app.

Scope rule

Any one of these can become a future enhancement or separate project. None is allowed to delay finishing Project #1.

20. Risks and How to Avoid Them

Risk	Failure Mode	Control
Overengineering the UI	Spending hours picking frameworks/themes instead of finishing.	Use Tkinter/ttk. Get the cards working before visual polish.
Adding too many metrics	Turning a one-weekend project into a monitoring suite.	Freeze MVP at four metrics.
Blocking the GUI	Using sleep/loops inside the main UI thread.	Use after(...) scheduling.
Packaging rabbit hole	Losing time on installers/signing.	A working executable is nice-to-have; source + clear instructions still counts if packaging becomes disproportionate.
Tutorial-copy behavior	Following a dashboard tutorial line-for-line.	Design the requirements first and understand every function used.
Stopping at “it works”	Leaving rough spacing, no README, debug code.	Reserve a full final session specifically for polish and closeout.

21. Learning Objectives

- Use Python comfortably enough to create a small real application.
- Install and manage a third-party dependency.
- Collect operating-system information through a library rather than hard-coded values.
- Understand the basic event-driven model of a desktop GUI.
- Update UI state repeatedly without freezing the app.
- Design clean boundaries between data collection and presentation.
- Practice structured debugging rather than random edits.
- Practice Git through meaningful project milestones.
- Package or distribute a small application.
- Perform a deliberate final polish pass.
- Write a README that demonstrates understanding rather than just listing commands.

22. Portfolio / Resume Positioning

This project should not be sold as an advanced systems monitor. Its value is that it is a finished, clean utility demonstrating basic systems integration and software craftsmanship. The strongest presentation is concise and factual.

Example portfolio description

Built a lightweight Windows system-monitoring desktop application in Python using psutil and Tkinter. Implemented live CPU, memory, disk, and uptime metrics with responsive event-driven updates, graceful error handling, and a polished single-window interface; packaged and documented the application for reproducible use.

23. Handoff to the Organization Manager

When formally creating the project inside the CSE organization system, provide the organization manager the following canonical project facts:

Project Field	Canonical Value
Name	PC System Dashboard
Classification	Serious project - small / first portfolio workflow test

Status	Planned -> Active when repo/build work begins
Objective	Build and finish a polished Windows-first desktop dashboard showing live CPU, memory, disk, and uptime metrics.
MVP	One window, four metric cards, automatic refresh, graceful unavailable states.
Primary stack	Python + psutil + Tkinter/ttk
Expected effort	5-8 focused hours
Completion gate	Functional + tested + polish pass + README + screenshot + clean repo + final version/tag
Do not expand	No GPU, charts, AI, remote monitoring, accounts, or process management in v1.

24. First Action When You Start Building

Start here

Create the repository and Python environment. Install psutil. Write a tiny metrics.py that prints CPU %, RAM %, disk %, and formatted uptime to the terminal. Do not build the GUI until those four values are correct. Then commit that working state.

This specification is intentionally strict about scope because Project #1 is meant to validate the full project system. Finishing the project cleanly is more valuable than making it technically large.